Lab 5: Static Code Analysis

Student Name: John S Thomas | SRN: PES2UG24CS207 | Course: Software Engineering Lab
Target Program: scholarship_disbursement.py | Repository: https://github.com/Prober55/Static-Code-Analysis

1. Objective & Overview

The objective of this laboratory is to evaluate and improve code quality, security, and PEP 8 style in an existing Python application (scholarship_disbursement.py) using static analysis tools: Pylint (code quality and design flaws), Bandit (security vulnerabilities and CWE detection), and Flake8 (PEP 8 style and syntax enforcement). All issues were systematically resolved, verified against baseline program execution, and committed to version control.

2. Tool Metrics: Initial vs. Cleaned Code

Tool / Metric	Initial Run	Final Run	Target / Standard	Status
Pylint Score	5.00 / 10.00 (16 issues)	10.00 / 10.00 (0 issues)	Score > 9.5 / 10.0	Passed (Perfect)
Bandit Security Scan	4 issues (1 Med, 3 Low)	0 issues identified	0 vulnerabilities	Passed (Clean)
Flake8 Style Check	6 PEP 8 violations	0 violations	PEP 8 Compliance	Passed (Clean)
Program Output	Baseline output (3 lines)	Identical output verified	Preserve behavior	Passed (Working)

3. Documented Issues & Remediation Table

Issue	Tool & Code	Type	Line	Description	Fix Approach
SQL Injection	Bandit B608	Security (Med)	10	String formatting in SQL INSERT query introduces SQL injection vector (CWE-89).	Replaced % formatting with parameterized placeholders (?, ?, ?, 0).
Mutable Default Arg	Pylint W0102	Bug / Defect	9	audit=[] default argument is shared across all function calls.	Changed default to None; initialized audit = [] inside grant().
Insecure Shell Exec	Bandit B605/7	Security (Low)	26	os.system('echo archiving awards') spawns subshell with partial path (CWE-78).	Replaced with native Python print() and removed unused os import.
Broad Except & Pass	Bandit B110, Pylint W0718	Quality	27-28	try-except Exception: pass silently swallows unexpected exceptions.	Removed unnecessary try-except block to expose runtime errors.
Singleton Comparison	Flake8 E711, Pylint C0121	Style / PEP 8	32	Used equality comparison scheme == None instead of identity comparison.	Changed check to scheme is None.
Function Naming	Pylint C0103	Style / PEP 8	31	bySchemeCount used camelCase instead of PEP 8 snake_case convention.	Renamed to by_scheme_count (with backward compatibility alias).
Duplicate Disbursement	Logic Defect, Pylint R1710	Correctness	17-20	Calling disburse() on already disbursed award re-added amount; inconsistent return.	Filtered WHERE id = ? AND disbursed = 0; added explicit return None.

Global State Mutation	Pylint W0603, C0103	Design / PEP 8	6, 16	global disbursed_total violates modular design and PEP 8 constant naming.	Direct SQL sum query via get_disbursed_total() and module __getattr__.
Formatting & Docs	Flake8 E501, Pylint C0114/6	Style / Docs	1-45	Missing module/function docstrings; line lengths exceeded 79 characters.	Added comprehensive docstrings; formatted lines under 79 chars.

4. Key Code Refactoring Highlights

Fix 1: Parameterized SQL Query (Bandit B608)
Before: conn.execute("INSERT INTO awards ... VALUES ('%s', '%s', %d, 0)" % (student, scheme, amount))
After: conn.execute("INSERT INTO awards ... VALUES (?, ?, ?, 0)", (student, scheme, amount))

Fix 2: Mutable Default Argument (Pylint W0102) #
Before: def grant(student, scheme, amount, audit=[]):
After: def grant(student, scheme, amount, audit=None): if audit is None: audit = []

Fix 3: Elimination of Insecure Shell & Silent Pass (Bandit B605/B607/B110)
Before: def archive(): try: os.system("echo archiving awards") except Exception: pass
After: def archive(): print("archiving awards")

Fix 4: Safe Disbursement & Database Aggregation (Pylint W0603/C0103 & Correctness)
Disburse only undisbursed awards: WHERE id = ? AND disbursed = 0
Disbursed sum: SELECT COALESCE(SUM(amount), 0) FROM awards WHERE disbursed = 1

5. Lab Reflection & Discussion

Q1. Purpose and Scope of Static Code Analysis:
Static code analysis inspects source code without execution by analyzing the Abstract Syntax Tree (AST), syntax, and structural patterns. Unlike dynamic testing, which only validates execution paths covered by test cases, static analysis shifts vulnerability and bug detection to the earliest stages of the SDLC. It detects high-risk security flaws (e.g., injection, insecure subshells), anti-patterns (mutable defaults), and maintainability issues before code is deployed.

Q2. Tool Roles & Complementary Strengths (Pylint vs. Flake8 vs. Bandit):
Each tool addresses a different dimension of software quality: Flake8 acts as a fast, strict linter enforcing PEP 8 styling, line lengths, and whitespace syntax; Pylint functions as a comprehensive code reviewer evaluating architectural smells, dangerous defaults, variable scoping, and docstrings with an objective quality score; Bandit acts as a specialized security auditor scanning AST nodes for Common Weakness Enumerations (CWEs). Using all three together ensures code is readable, architecturally sound, and secure.

Q3. Challenges Encountered & Remediation Strategy:
The primary challenge was resolving structural warnings without altering functional contracts or masking defects. Rather than using warning suppressions (e.g., # pylint: disable), the underlying root causes were refactored: replacing global state with database aggregation, handling idempotency in award disbursements to prevent double payments, and preserving backward compatibility for camelCase callers (bySchemeCount) and module attributes (disbursed_total).

Q4. CI/CD Integration & Best Practices:
In modern DevOps pipelines, static analysis tools should be configured as automated quality gates within pre-commit hooks and CI workflows (e.g., GitHub Actions). Pull requests failing security (Bandit) or threshold quality scores (Pylint > 9.5) are automatically blocked. This eliminates subjective style debates during peer reviews, maintains uniform code standards across distributed teams, and prevents security regressions from reaching production.

6. Final Execution & Output Verification

The refactored script was executed in terminal (python3 scholarship_disbursement.py) and generated the exact verified output matching baseline behavior:

```
50000 None 1
50000 archiving
awards
```

Git Repository status is clean with all incremental commits pushed to https://github.com/Prober55/Static-Code-Analysis.

